



2026-2학기 데이터사이언스

6강. 깃과 깃허브 기록하고 나누기

2026. 10. 12.

분석_최종_진짜최종_v3.py

- 어제 잘 되던 분석이 오늘 안 된다. 무엇을 바꿨더라
- AI 가 파일을 고쳤는데 되돌리고 싶다
- 제출한 결과를 남이 그대로 재현할 수 있어야 한다

AI와 함께 일할수록 더 필요해진다

클로드는 파일을 실제로 고친다.

고치기 전 상태가 어딘가 남아 있어야 마음 놓고 시킬 수 있다

→ 깃은 되돌릴 수 있게 해 주고, 깃허브는 그것을 남에게 보여 줄 수 있게 한다.
증거를 다루는 사람에게 이것은 편의가 아니라 **요건**이다

오늘 배울 세 가지

무엇	해결하는 것	이 수업에서의 쓸모
① 깃	내 컴퓨터 안의 기록과 되돌리기	AI 가 고친 것을 되돌린다
② 깃허브	남과 나누기 와 백업	분석을 제출하고 재현시킨다
③ 클로드 + 깃	커밋·PR 을 맡기기	기록을 남기는 품을 줄인다

깃 명령을 외우는 수업이 아니다. **무엇을 왜 기록하는지**를 배우고 명령은 클로드에게 맡긴다.

저장한다 ≠ 기록한다

그냥 저장하면



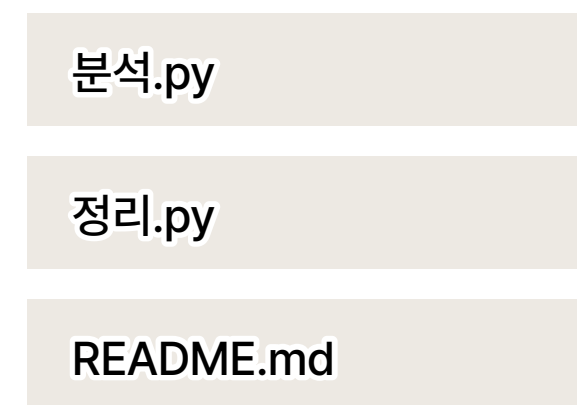
커밋을 쌓으면



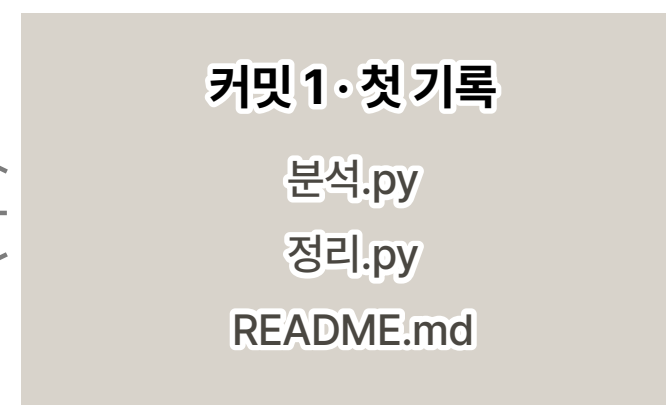
커밋은 "이 시점의 전부" 를 이름과 함께 남긴다

깃은 바뀐 것만 골라 쌓는다

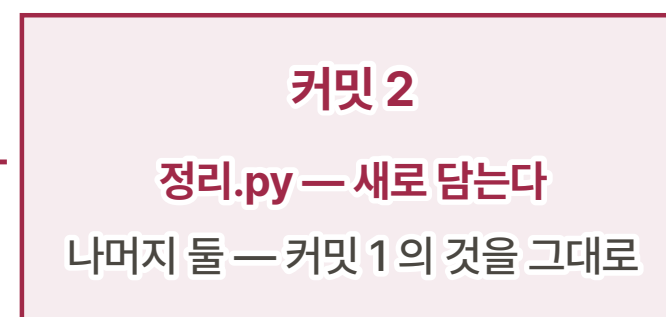
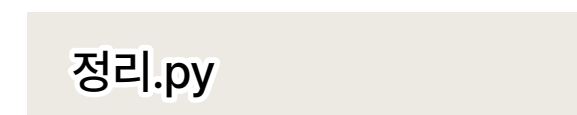
작업 폴더



커밋



정리.py 한 줄을 고치고 다시 커밋하면



깃은 파일 이름이 아니라 **내용을 보고** 같은지 다른지 안다.

그래서 수천 개 파일 중 바뀐 것만 골라 담고,
나머지는 앞의 것을 다시 쓴다.

그래서 커밋이 쌓여도
저장소가 그만큼 무거워지지 않는다.

일단 이 넷이면 된다

터미널

`git init`이 폴더를 기록 대상으로

`git status`지금 뭐가 바뀌었나

`git add .`기록할 것을 고른다

`git commit`이름 붙여 남긴다
`-m "동별 요약 로직 확정"`

`git log --oneline`지금까지의 기록

`git diff`무엇이 바뀌었나

되돌리는 명령은 위험해서 여기 적지 않았다. **필요할 때 클로드에게 물어본다** — 무엇이 지워지는지 설명을 듣고 승인한다.

커밋 메시지를 이렇게 쓴다

이렇게 말고	이렇게
수정	면적 하한을 60m ² 로 정함
□ L O R	취소 거래 제외 규칙 추가
최종	동별 요약표 생성 로직

"무엇을 왜 바꿨는지" 를 적는다. 석 달 뒤에 이 목록만 읽고도 분석의 흐름이 보여야 한다.

넣지 않을 것 — .gitignore

넣지 않는다	왜
인증키 · 비밀번호	한 번 올라가면 지워도 기록에 남는다. 저장소를 공개로 바꾸는 순간 노출된다
큰 원본 데이터	수백 MB 를 커밋하면 저장소가 감당하지 못한다. <u>받는 방법을 적어 두면 된다</u>
개인정보가 든 자료	3주차에 말한 비공개 데이터가 여기 해당한다
다시 만들 수 있는 것	그림·PDF 처럼 스크립트로 재생성되는 산출물

.gitignore

.env

인증키

data/raw/

원본 데이터 — README 에 받는 법을 적는다

output/*.png

스크립트로 다시 만든다

첫 커밋 전에 .gitignore 부터 만든다. 올라간 뒤에 지우는 것은 훨씬 어렵다.

내 컴퓨터 밖에 두기

깃은 내 컴퓨터 안의 기록이다. 노트북이 고장 나면 함께 사라진다.

깃허브는 그 기록을 인터넷에 올려 두는 곳이다. 백업이자, 남에게 보여 주는 창구다.

터미널 · gh 를 설치하고 로그인한 뒤

gh auth login	한 번만
gh repo create	저장소 만들기
git push	올리기
git pull	내려받기

공개할 것과 하지 않을 것

이 수업이라면	
공개(public)	분석 스크립트 · 로직 문서 · 결과표 · README
비공개(private)	확정 전 초안, 개인정보가 섞였을 가능성이 있는 것

공개는 되돌리기 어렵다. 헛갈리면 비공개로 만들고 나중에 바꾼다 — 반대 방향은 이미 늦다.

COMPAS 비공개 데이터는 어느 쪽에도 올리지 않는다.

README — 재현의 설명서

남이 이 저장소를 열었을 때 **가장 먼저 보는 화면**이다.

이 수업의 기준은 하나다 — **이것만 읽고 내 결과를 다시 만들 수 있는가.**

원본 데이터를 올리지 않는 대신 **받는 방법을 적는다.** 그것이 3주차 "데이터 출처 문서"가 여기로 오는 자리다.

README.md

청주 상당구 실거래가 분석

무엇을 했나

동별 m²당 가격을 비교했다.

데이터

국토부 실거래가 API (공공데이터포털)

상당구 43111 · 2025.10~2026.09

받는 법: 인증키를 발급받아

`env`에 넣고 `python fetch.py`

분석 규칙

CLAUDE.md 참조 (60m² 초과 · 중앙값)

다시 만들려면

`python fetch.py` → `python summary.py`

결과는 `output/`에

기록하는 품을 맡긴다

클로드 코드는 터미널에서 돌므로 깃 명령을 그대로 쓸 수 있다

이렇게 시킨다

"지금까지 바뀐 걸 무엇을 왜 바꿨는지 알 수 있게 커밋해 줘. 커밋하기 전에 무엇을 담을지 목록으로 보여 줘."

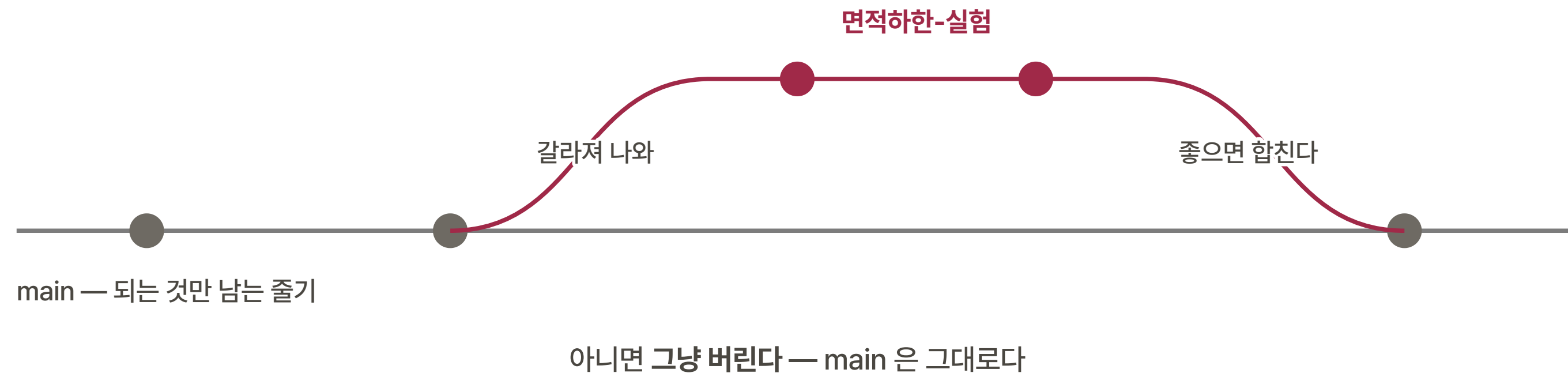
".env 는 빼고 커밋해."

"이 저장소의 커밋 기록을 읽고 분석이 어떻게 바뀌어 왔는지 정리해 줘."

굵게 표시한 부분이 핵심이다. **커밋 전에 목록을 보여 달라고 한다** — 무엇이 올라가는지 모르고 승인하면 인증키가 함께 간다.

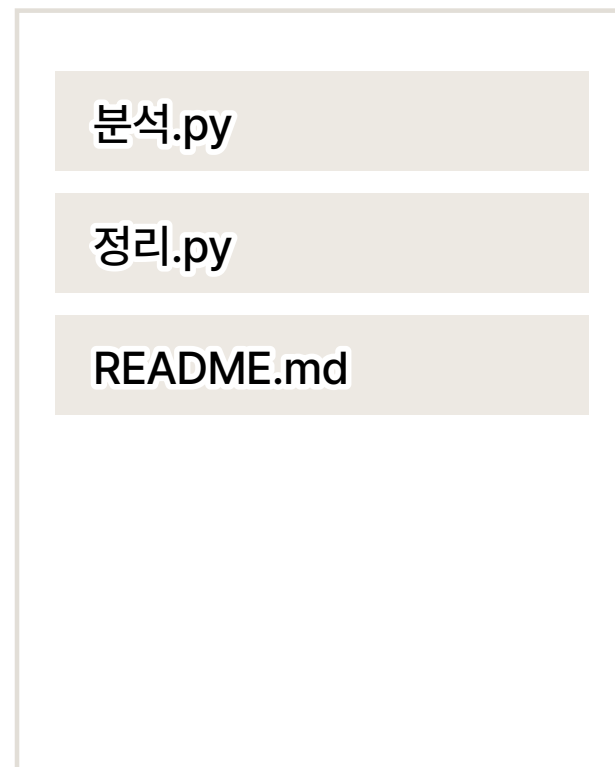
마지막 줄은 이 수업다운 쓰임이다 — 커밋 기록 자체가 **분석 과정의 증거**가 된다.

브랜치 — 망칠 각오로 해 보기



깃에는 자리가 넷 있다

작업 폴더



커밋 대기



저장소 — 내 컴퓨터

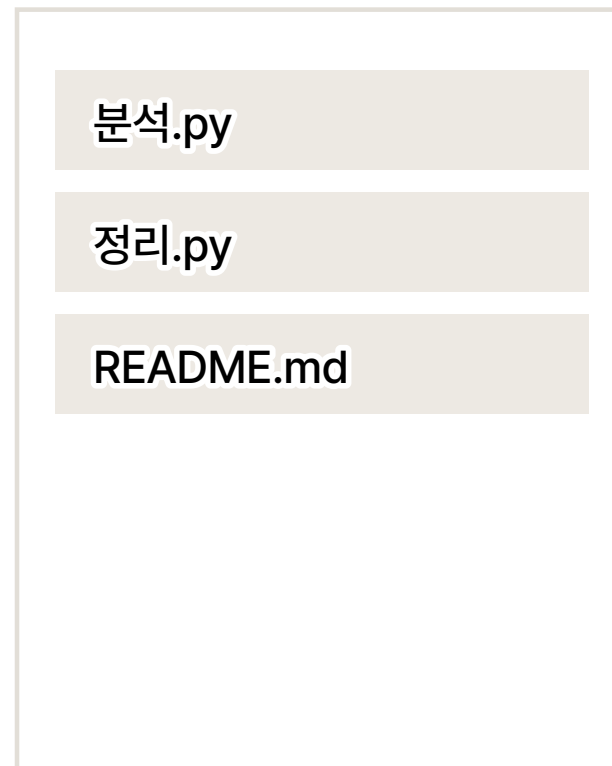


깃허브 — 인터넷



깃에는 자리가 넷 있다

작업 폴더



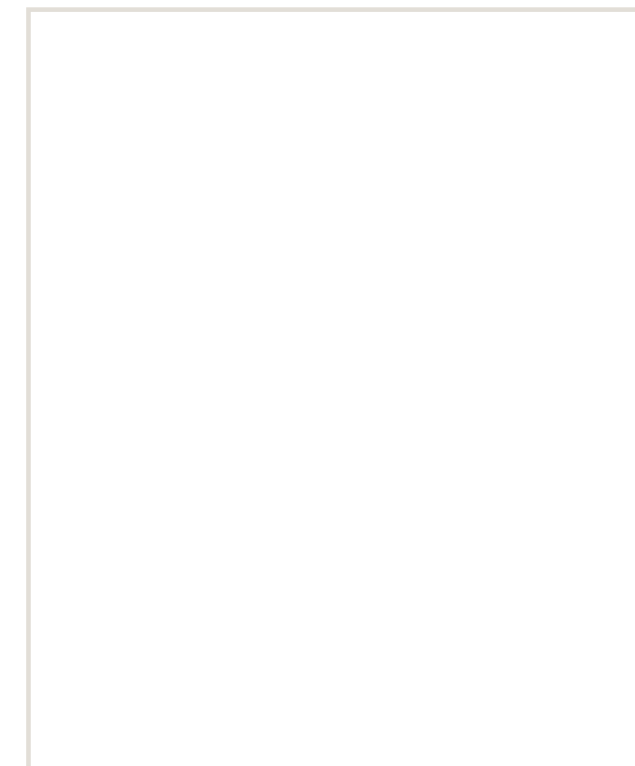
커밋 대기



저장소 — 내 컴퓨터



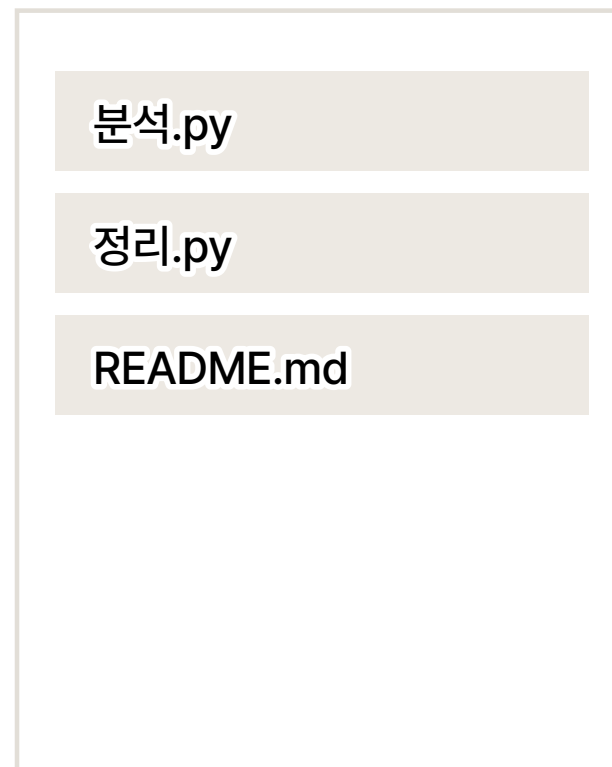
깃허브 — 인터넷



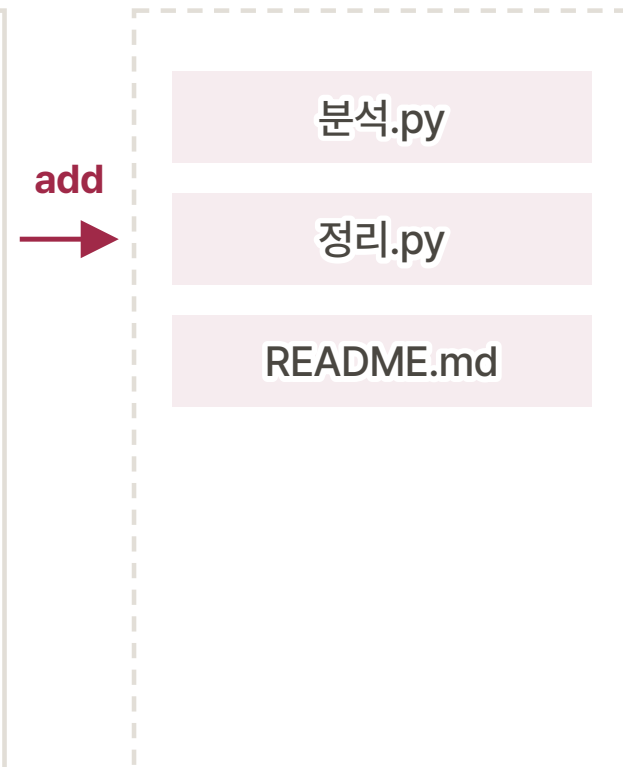
git init — 이 폴더를 기록 대상으로 삼는다

깃에는 자리가 넷 있다

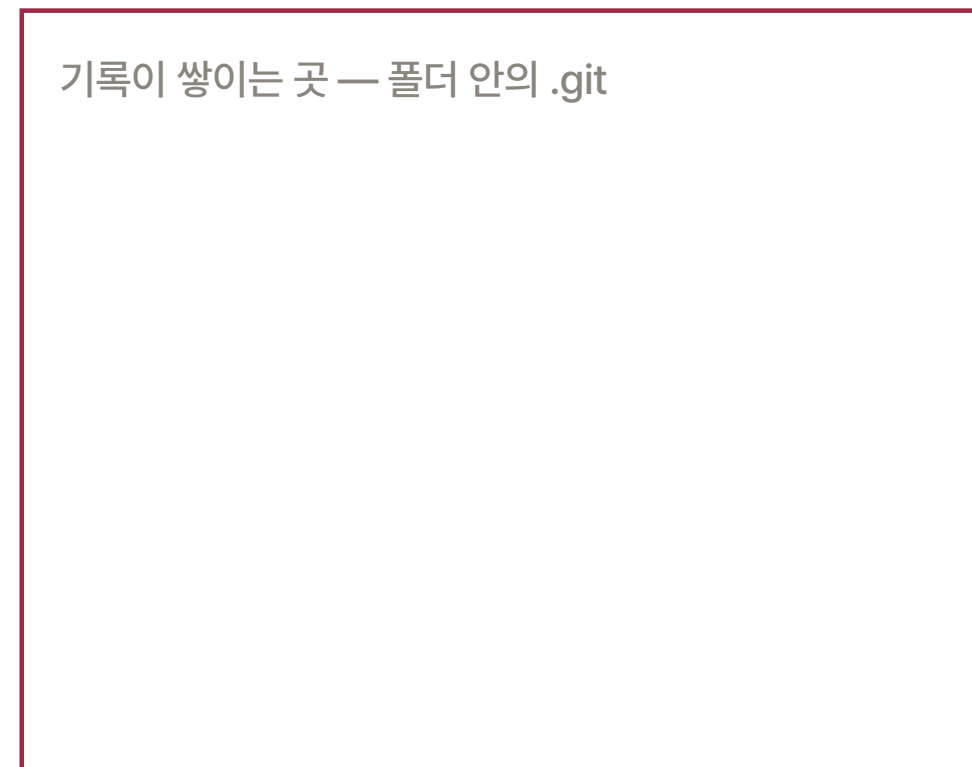
작업 폴더



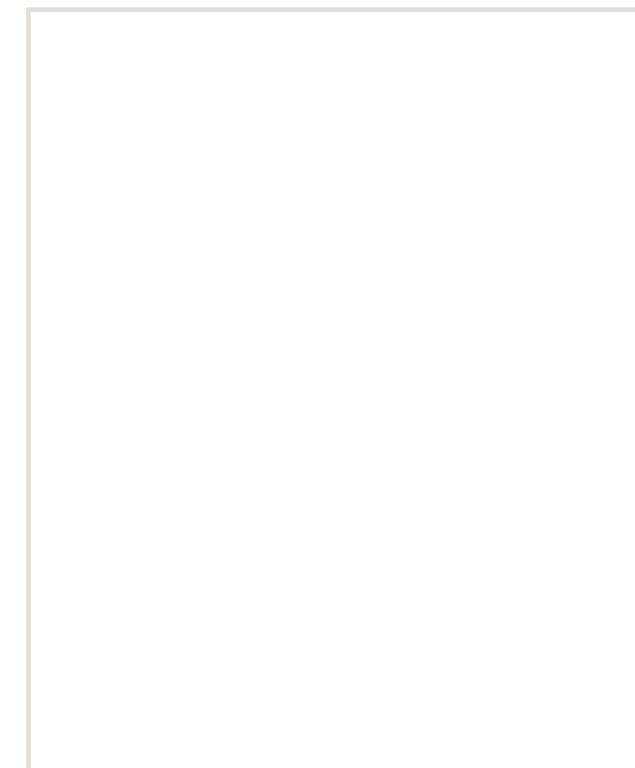
커밋 대기



저장소 — 내 컴퓨터



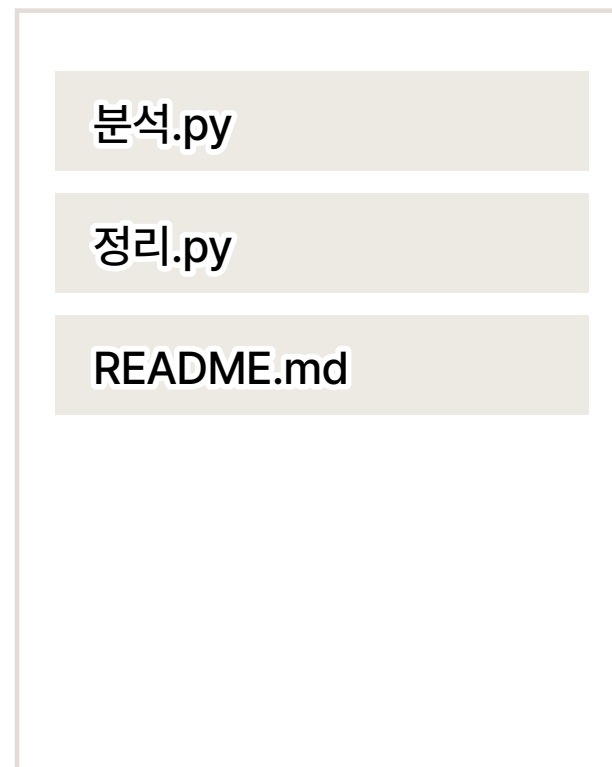
깃허브 — 인터넷



git add . — 이번 커밋에 담을 것을 골라 둔다. 파일은 그 자리에 그대로 있다

깃에는 자리가 넷 있다

작업 폴더



커밋 대기

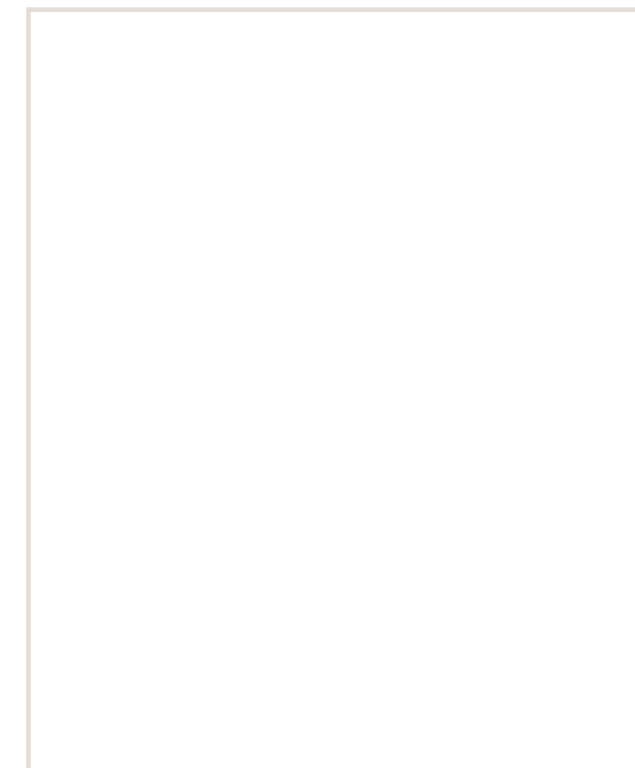


저장소 — 내 컴퓨터

commit

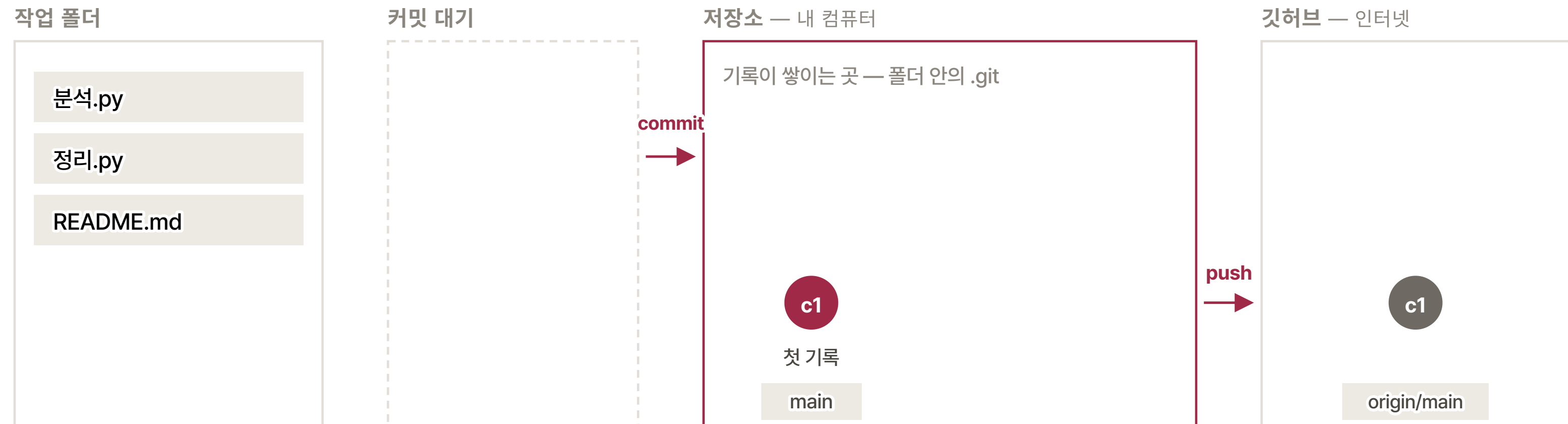


깃허브 — 인터넷



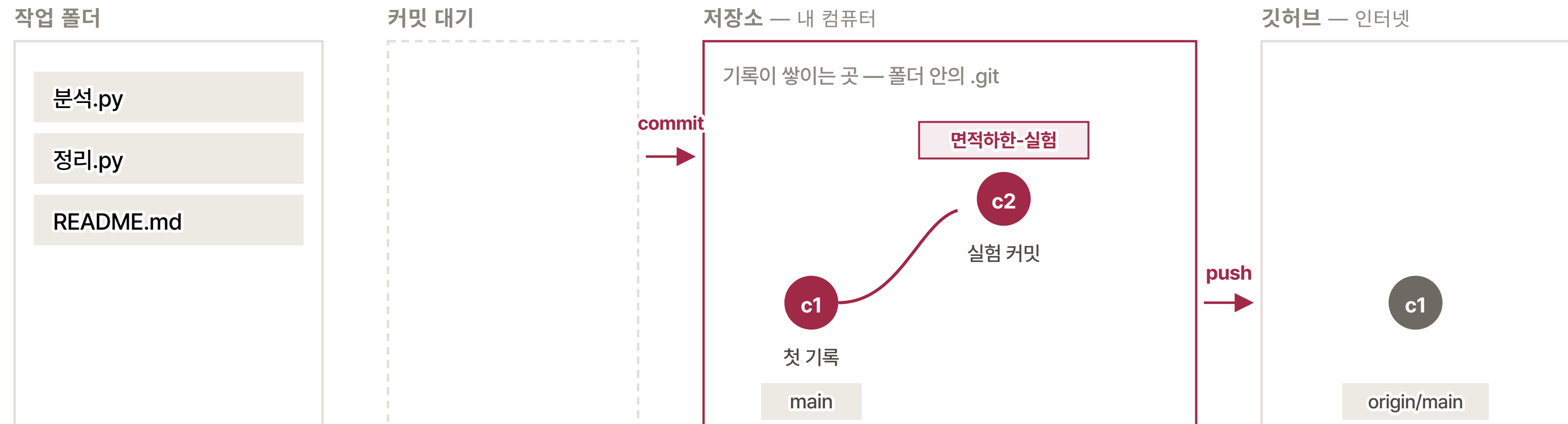
git commit -m "첫 기록" — 골라 둔 것을 이름 붙여 저장소에 남긴다

깃에는 자리가 넷 있다



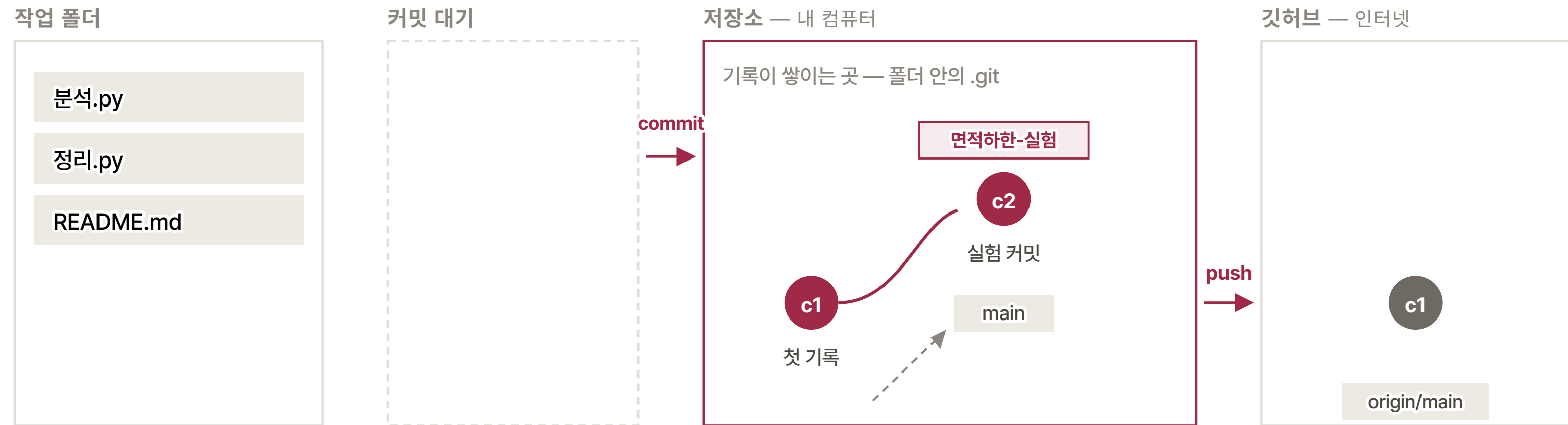
git push — 저장소의 커밋을 깃허브로 올린다

깃에는 자리가 넷 있다



`git switch -c 면적하한-실험` — `main` 을 두고 딴 줄기로 나가 커밋한다

깃에는 자리가 넷 있다



main 이 실험 자리까지 따라온다 — 새 커밋 없이 포인터만 옮겨 간다

git merge 면적하한-실험 — 실험이 쓸 만하면 main 으로 합친다

PR — 합치기 전에 보여 주기

풀 리퀘스트(PR)는 **"이렇게 바꾸려는데 봐 주세요"** 라는 요청이다. 바뀐 부분이 한눈에 보이고, 줄마다 의견을 달 수 있다.

터미널

```
git switch -c 면적하한-실험
# ... 작업 ...
git push -u origin 면적하한-실험
gh pr create
```

클로드에게 "브랜치 만들고 PR 까지 올려 줘" 라고 해도 된다. **올리기 전에 무엇을 담을지 확인한다.**

깃허브에서 클로드를 부르기

`/install-github-app` 을 실행해 두면 이슈나 PR 댓글에 **@claude** 로 부를 수 있다.

PR 댓글

@claude 이 분석 로직에서
빠뜨린 예외가 있는지 봐 줘

GitHub Actions 위에서 돌기 때문에 **저장소 관리자 권한**과 인증 설정이 필요하다. 팀으로 일할 때 쓰는 기능이라 오늘은 개념만 본다.

주의 — 이 수업에서 실제로 겪는 사고

사고	막는 법
인증키를 커밋했다	첫 커밋 전에 <code>.gitignore</code> 부터 만든다. 이미 올렸다면 그 키를 폐기하고 새로 발급받는다 — 기록에서 지우는 것보다 그게 빠르고 확실하다
비공개 데이터를 올렸다	COMPAS 비공개분·개인정보는 저장소에 넣지 않는다. <u>받는 방법만 적는다</u>
큰 파일로 저장소가 막혔다	원본 데이터는 <code>.gitignore</code> 로 뺀다
AI 가 고친 걸 못 되돌린다	작업을 시작하기 전에 커밋해 둔다. 되돌릴 지점이 있어야 마음 놓고 시킨다

첫 줄이 가장 흔하고 가장 아프다. **공개 저장소는 검색된다.**

실습

내 분석을 저장소로 만들기 — 약 1시간 30분

오늘 실습의 도착점

남이 README 만 보고 내 결과를 다시 만들 수 있는 저장소

시각	할 일	끝났다는 신호
0:00–0:25	① 기록을 시작한다	첫 커밋이 남았다
0:25–0:55	② 브랜치로 실험하고 되돌린다	main 이 그대로 다
0:55–1:30	③ 서로 재현해 본다	남의 결과를 내가 만들었다

시작 전 확인

- 지난주 ds2026/ 폴더 (CLAUDE.md · 스킬 · 에이전트가 들어 있다)
- 깃 설치 · 깃허브 계정 · gh 설치

③은 짝이 필요하다. 홀수면 셋이 한 조로 돌려 가며 한다.

① 기록을 시작한다 0:00-0:25

- 1 `gh auth login` 으로 로그인한다 (브라우저가 열린다) 5분
- 2 `ds2026/` 에서 `git init` 1분
- 3 커밋하기 전에 `.gitignore` 를 만든다 8분
- 4 `git status` 로 올라갈 목록을 눈으로 읽는다 3분
- 5 `git add .` → `git commit -m "..."` 3분

3번을 이렇게 시킨다

"이 폴더를 깃 저장소로 만들려고 해. 먼저 어떤 파일이 올라가면 안 되는지 목록으로 알려 주고 `.gitignore` 초안을 보여 줘. 내가 확인하면 커밋하자."

✓ 확인 지점 — `git status` 목록에 인증키·원본 CSV 가 없다. 보이면 `.gitignore` 를 고치고 다시 본다.

② 실험하고 되돌린다 0:25-0:55

- 1 `git switch -c 면적하한-실험` 2분
- 2 CLAUDE.md 의 면적 하한을 **85m² 로 바꾸고** 분석을 다시 돌린다 10분
- 3 `git diff` 로 무엇이 바뀌었는지 보고 커밋한다 5분
- 4 `git switch main` 으로 돌아간다 2분
- 5 CLAUDE.md 를 열어 본다. 분석도 다시 돌려 본다 6분
- 6 실험을 살릴지 버릴지 정하고, 버릴 거면 브랜치를 지운다 5분

✓ 확인 지점 — 5번에서 CLAUDE.md 가 60m² 로 돌아와 있다. 결과표도 원래 값이다. 실험이 본 작업을 건드리지 않았다.

switch 가 거부되면 — 저장하지 않은 변경이 있는 것이다. 커밋하거나 되돌린 뒤 다시 한다. 무엇을 어떻게 할지는 클로드에게 물어보고 설명을 듣고 승인한다.

③-1 올리고 설명서를 쓴다 0:55-1:15

- 1 `gh repo create` 로 저장소를 만들고 올린다. **공개 여부를 스스로 정하고 이유를 한 줄 적는다** 7분
- 2 `README.md` 를 쓴다 10분
- 3 커밋하고 올린다. 브라우저로 저장소를 열어 눈으로 확인한다 3분

README 에 반드시 들어갈 넷

무엇을 했나	한두 문장
데이터 받는 법	출처 · 조건 · 인증키를 어디에 넣나 (3주차 출처 문서를 옮긴다)
분석 규칙	CLAUDE.md 를 가리키고 요점만
다시 만드는 순서	실행할 명령을 순서대로

✓ **확인 지점** — 브라우저에서 저장소를 열었을 때 **인증키도 원본 데이터도 보이지 않는다.**

③-2 서로 재현해 본다 1:15-1:30

- 1 옆 사람과 저장소 주소를 바꾼다 1분
- 2 상대 저장소를 받는다 — `git clone <주소>` 2분
- 3 **README 만 보고** 상대의 결과를 다시 만들어 본다. 막히면 물어보지 말고 그 지점을 적는다 8분
- 4 막힌 지점을 상대에게 알려 준다. 받은 지적으로 자기 README 를 고친다 4분

✓ 확인 지점 — 상대의 결과표와 **같은 숫자**가 나온다. 안 나오면 README 에 빠진 것이 있다는 뜻이고, **그 발견이** 오늘의 성과다.

시간이 모자라면 — 3번의 막힌 지점 목록까지만 하고 4번은 집에서. 일찍 끝났으면 상대 저장소에 브랜치를 만들어 README 를 고치고 PR 을 올려 본다.

제출물

무엇	어느 정도면 되나
저장소 주소	공개 여부와 그렇게 정한 이유 한 줄
커밋 기록	<code>git log --oneline</code> 결과. 메시지만 읽고 흐름이 보여야 한다
README	넷이 다 있고 ③-2 로 고친 흔적 이 있을 것
재현 후기	상대 저장소에서 어디서 막혔는지 와 내 README 를 어떻게 고쳤는지

마지막 줄이 이 강의의 핵심이다. **내가 아는 것과 적어 둔 것은 다르다** — 남이 막히는 자리에서만 그 차이가 드러난다.

다음 주부터 매주 실습 결과를 이 저장소에 커밋한다. 14주차에는 이 기록 전체가 증거 패키지의 바탕이 된다.

자료 출처

- 깃 명령 — 공식 문서 `git-scm.com/doc`
- 깃허브 CLI — `cli.github.com`
- `/install-github-app` 과 `@claude` 멘션 — 클로드 코드 공식 문서 (2026. 8. 27. 확인)
`code.claude.com/docs/en/github-actions` · 저장소 관리자 권한과 인증 설정이 필요하다
- 5·11쪽 그림은 원리를 보이는 개념도다

Q & A

권규상 Kyusang Kwon
kyusang.kwon@chungbuk.ac.kr